

=====

ПОЛНАЯ КОДОВАЯ БАЗА СТРОИТЕЛЬНОГО ТРЕКЕРА (АРХИТЕКТУРА JSON)

=====

ФАЙЛ: wizard_manager.py

```
import db_core
import wizard_menus
import git_deployer

def print_data():
    db = db_core.load_db()
    print("\n=== ТЕКУЩАЯ БАЗА ДАННЫХ ПРОЕКТА ===")
    if not db:
        print("База данных пуста.")
        return
    for direction, sub_objs in db.items():
        print(f"\n Направление: {direction}")
        for sub_obj, months in sub_objs.items():
            print(f"    └ Подобъект: {sub_obj}")
            for mth, data in months.items():
                print(f"        └ {mth}: {db_core.fmt_money(data.get('sum', 0))} руб. | Статус СтрК: {data.get('status', 'Нет')}")
    print("=====")

def run_wizard():
    while True:
        print("\n--- СМАРТ-ПОМОЩНИК УПРАВЛЕНИЯ ТРЕКЕРОМ (JSON БАЗА) ---")
        print("1. Посмотреть текущую базу данных (Дерево)")
        print("2. Добавить новую ветку структуры (Объект/Месяц)")
        print("3. Внести изменения в таблицу")
        print("4. Сгенерировать Excel (с запросом отправки на GitHub)")
        print("5. Выйти из помощника")
        choice = input("Выберите действие (1-5): ").strip()

        if choice == "1":
            print_data()
        elif choice == "2":
            wizard_menus.menu_add_record()
        elif choice == "3":
            wizard_menus.menu_edit_data()
        elif choice == "4":
            git_deployer.run_production_pipeline(session_added_rows=["Обновление базы данных трекера"])
        elif choice == "5":
            print("До свидания!"); break
        else:
            print(" Неверный ввод.")

if __name__ == "__main__":
    run_wizard()
```

ФАЙЛ: wizard_menus.py

```
import db_core

def select_hierarchy_target(db, allow_new=False):
    """Универсальный кнопочный выбор Направления и Подобъекта по номерам."""
    target_dir = ""
    directions = list(db.keys())

    if directions:
        print("\nДоступные Направления:")
        for idx, d in enumerate(directions, 1): print(f" {idx}. {d}")
        if allow_new: print(f" {len(directions) + 1}. [Создать абсолютно НОВОЕ Направление руками]")

        limit = len(directions) + 1 if allow_new else len(directions)
        d_choice = input(f"\nВыберите номер Направления (1-{limit}): ").strip()
        if not d_choice or not d_choice.isdigit(): return None, None
        d_idx = int(d_choice) - 1

        if 0 <= d_idx < len(directions): target_dir = directions[d_idx]
        elif allow_new and d_idx == len(directions):
            target_dir = input("Введите название НОВОГО Направления: ").strip()
    else:
        target_dir = input("База пуста. Введите название Направления: ").strip()
```

```

if not target_dir: return None, None

target_sub = ""
if target_dir in db and db[target_dir]:
    sub_objs = list(db[target_dir].keys())
    print(f"\nДоступные Подобъекты для '{target_dir}':")
    for idx, s in enumerate(sub_objs, 1): print(f" {idx}. {s}")
    if allow_new: print(f" {len(sub_objs) + 1}. [Создать абсолютно НОВОЙ Подобъект руками]")

    limit = len(sub_objs) + 1 if allow_new else len(sub_objs)
    s_choice = input(f"Выберите номер Подобъекта (1-{limit}): ").strip()
    if not s_choice or not s_choice.isdigit(): return None, None
    s_idx = int(s_choice) - 1

    if 0 <= s_idx < len(sub_objs): target_sub = sub_objs[s_idx]
    elif allow_new and s_idx == len(sub_objs):
        target_sub = input("Введите название НОВОГО Подобъекта: ").strip()
else:
    target_sub = input("Введите название Подобъекта: ").strip()

return target_dir, target_sub

def menu_add_record():
    """Логика добавления новой ветки (Пункт 2)."""
    print("\n>> ДОБАВЛЕНИЕ НОВОЙ ВЕТКИ СТРУКТУРЫ")
    db = db_core.load_db()

    target_dir, target_sub = select_hierarchy_target(db, allow_new=True)
    if not target_dir or not target_sub: return

    print("\nВыберите Месяц из календаря:")
    active_months = list(db[target_dir][target_sub].keys()) if (target_dir in db and target_sub in db[target_dir]) else []
    for i, m_name in enumerate(db_core.ALL_YEAR_MONTHS, 1):
        marker = " [УЖЕ ЕСТЬ]" if m_name in active_months else ""
        print(f" {i}. {m_name}{marker}")

    mth_choice = input("\nВведите номер месяца (1-12): ").strip()
    if not mth_choice or not mth_choice.isdigit(): return
    mth_idx = int(mth_choice) - 1
    if mth_idx < 0 or mth_idx >= 12: return
    month = db_core.ALL_YEAR_MONTHS[mth_idx]

    if target_dir not in db: db[target_dir] = {}
    if target_sub not in db[target_dir]: db[target_dir][target_sub] = {}
    if month in db[target_dir][target_sub]: return

    db[target_dir][target_sub][month] = {"sum": 0.0, "status": {"value": "", "date": ""}}
    db_core.save_db(db)
    db_core.update_config_months(month)
    print(f" [УСПЕХ] Период '{month}' успешно добавлен!")

def menu_edit_data():
    """Логика изменения бюджетов, месяцев и статусов (Пункт 3)."""
    print("\n>> ВНЕСЕНИЕ ИЗМЕНЕНИЙ В ТАБЛИЦУ")
    db = db_core.load_db()
    if not db: return

    print("Выберите тип изменений:\n 1. Изменить сумму месяца\n 2. Переименовать месяц\n 3. Изменить статус СтрК")
    sub_choice = input("Выберите действие (1-3): ").strip()
    if sub_choice not in ("1", "2", "3"): return

    target_dir, target_sub = select_hierarchy_target(db, allow_new=False)
    if not target_dir or not target_sub: return

    months_in_db = list(db[target_dir][target_sub].keys())
    print(f"\nТекущие активные периоды:")
    for idx, m in enumerate(months_in_db, 1):
        st_raw = db[target_dir][target_sub][m].get("status", "")
        st_val = st_raw.get("value", "Нет") if isinstance(st_raw, dict) else (st_raw if st_raw else "Нет")
        print(f" {idx}. {m} (Сумма: {db_core.fmt_money(db[target_dir][target_sub][m].get('sum', 0))} руб. | СтрК: {st_val})")

    m_choice = input("\nВыберите номер периода: ").strip()
    if not m_choice or not m_choice.isdigit(): return
    m_idx = int(m_choice) - 1
    if m_idx < 0 or m_idx >= len(months_in_db): return
    target_mth = months_in_db[m_idx]

```

```

if sub_choice == "1":
    new_sum_str = input(f"\nВведите новую сумму: ").strip()
    try:
        db[target_dir][target_sub][target_mth]["sum"] = float(new_sum_str.replace(",", "."))
        db_core.save_db(db)
        print(" [УСПЕХ] Бюджет обновлен.")
    except: print(" [ОШИБКА] Неверное число.")

elif sub_choice == "2":
    print("\nВыберите НОВОЕ название месяца:")
    for i, m_name in enumerate(db_core.ALL_YEAR_MONTHS, 1): print(f" {i}. {m_name}")
    new_mth_choice = input("\nВведите номер (1-12): ").strip()
    if not new_mth_choice or not new_mth_choice.isdigit(): return
    new_mth_name = db_core.ALL_YEAR_MONTHS[int(new_mth_choice) - 1]

    if new_mth_name in db[target_dir][target_sub]:
        print(" ⚠ Месяц уже существует! Объединяем? (1-Да / 2-Нет)")
        if input().strip() == "1":
            old = db[target_dir][target_sub].pop(target_mth)
            db[target_dir][target_sub][new_mth_name]["sum"] += float(old.get("sum", 0))
            db_core.save_db(db)
        return

    db[target_dir][target_sub][new_mth_name] = db[target_dir][target_sub].pop(target_mth)
    db_core.save_db(db)
    db_core.update_config_months(new_mth_name)

elif sub_choice == "3":
    print("\nВыберите статус СтрК:\n 1. Зеленый | 2. Желтый | 3. Красный | 0. Очистить")
    st_choice = input("Введите номер (0-3): ").strip()
    if st_choice in ("1", "2", "3"):
        db[target_dir][target_sub][target_mth]["status"] = {"value": int(st_choice), "date":
db_core.get_short_date()}
        db_core.save_db(db)
        print(" [УСПЕХ] Статус обновлен.")
    elif st_choice == "0":
        db[target_dir][target_sub][target_mth]["status"] = {"value": "", "date": ""}
        db_core.save_db(db)
        print(" [УСПЕХ] Статус очищен.")

```

ФАЙЛ: db_core.py

```

import os
import json
import datetime
import config

ALL_YEAR_MONTHS = [
    "Январь", "Февраль", "Март", "Апрель", "Май", "Июнь",
    "Июль", "Август", "Сентябрь", "Октябрь", "Ноябрь", "Декабрь"
]

def fmt_money(value):
    """Форматирует число с пробелами и русской запятой для копеек."""
    try:
        val_float = float(value)
        if val_float.is_integer():
            return f"{int(val_float):_}".replace("_", " ")
        else:
            parts = f"{val_float:.2f}".split(".")
            thousand_part = f"{int(parts):_}".replace("_", " ")
            return f"{thousand_part},{parts}"
    except:
        return str(value)

def get_short_date():
    """Возвращает текущую дату в формате ДД.ММ"""
    return datetime.datetime.now().strftime("%d.%m")

def load_db():
    json_path = "database.json"
    if os.path.exists(json_path):
        with open(json_path, "r", encoding="utf-8") as f:
            return json.load(f)
    return {}

def save_db(db):
    json_path = "database.json"

```

```

with open(json_path, "w", encoding="utf-8") as f:
    json.dump(db, f, ensure_ascii=False, indent=4)

def update_config_months(new_month):
    """Автоматически добавляет новый месяц в config.py с сохранением хронологии."""
    if new_month not in config.MONTHS_LIST:
        config_path = "config.py"
        with open(config_path, "r", encoding="utf-8") as f:
            code = f.read()

        updated_months = [m for m in ALL_YEAR_MONTHS if m in config.MONTHS_LIST or m == new_month]
        old_line_repr = f"MONTHS_LIST = {repr(config.MONTHS_LIST)}"
        new_line_repr = f"MONTHS_LIST = {repr(updated_months)}"

        if old_line_repr in code:
            code = code.replace(old_line_repr, new_line_repr)
        else:
            import re
            code = re.sub(r"MONTHS_LIST\s*=\s*\[.*?\]", new_line_repr, code)

        with open(config_path, "w", encoding="utf-8") as f:
            f.write(code)
        print(f" [СИНХРОНИЗАЦИЯ] Месяц '{new_month}' добавлен в хронологический список config.py")

```

ФАЙЛ: builder.py

```

import openpyxl
import json
import os
import datetime
import config
import excel_styler

def apply_row_style(ws, row_idx, font, fill, border, alignment=None):
    """Прокси-функция для сохранения совместимости со старым вызовом из main.py"""
    excel_styler.apply_row_style(ws, row_idx, font, fill, border, alignment)

def build_structure(ws, mock_data=None, saved_statuses=None, saved_sums=None):
    """Генерирует структуру табличной части с внедрением 5-го уровня для дат изменений."""
    ws.freeze_panes = "A2"
    ws.sheet_properties.outlinePr.summaryBelow = False

    now_str = datetime.datetime.now().strftime("%d.%m.%Y %H:%M")
    ws.oddHeader.right.text = f"Выгружено: {now_str}"
    ws.oddHeader.right.size = 9
    ws.oddHeader.right.color = "7A7A7A"

    if ws.max_row == 1 and ws.cell(row=1, column=1).value is None:
        ws.append(config.HEADERS)
        excel_styler.apply_row_style(ws, 1, config.FONT_HDR, config.FILL_HDR, config.THIN_BORDER,
config.ALIGN_C)

    json_path = "database.json"
    if not os.path.exists(json_path): return ws.max_row

    with open(json_path, "r", encoding="utf-8") as f:
        db = json.load(f)

    row_counter = 1

    for direction in db.keys():
        ws.append(["", str(direction), "", "", "", "", "", "", "", ""])
        excel_styler.apply_row_style(ws, ws.max_row, config.FONT_DIR, config.FILL_DIR,
config.THIN_BORDER, config.ALIGN_L)
        ws.row_dimensions[ws.max_row].outline_level = 0

        for sub_obj in db[direction].keys():
            ws.append(["", str(sub_obj), "", "", "", "", "", "", "", ""])
            excel_styler.apply_row_style(ws, ws.max_row, config.FONT_OBJ, config.FILL_OBJ,
config.THIN_BORDER, config.ALIGN_L)
            ws.row_dimensions[ws.max_row].outline_level = 1

            for mth in config.MONTHS_LIST:
                if mth not in db[direction][sub_obj]: continue
                mth_data = db[direction][sub_obj][mth]

                # Читаем дату из структуры статуса заранее
                status_raw = mth_data.get("status", "")

```

```

        status_date = status_raw.get("date", "") if isinstance(status_raw, dict) else ""

        # Месяц: Уровень 2
        ws.append([row_counter, mth, float(mth_data.get("sum", 0.0)), "", "", "", "", "", "", "",
""])

        current_row = ws.max_row
        excel_styler.apply_row_style(ws, current_row, config.FONT_MTH, config.FILL_MTH,
config.THIN_BORDER, config.ALIGN_L)
        ws.cell(row=current_row, column=1).alignment = config.ALIGN_C
        ws.cell(row=current_row, column=3).number_format = '#,##0.00'
        ws.row_dimensions[current_row].outline_level = 2
        row_counter += 1

        # Документы: Уровень 3
        for d_name in config.DOCUMENTS_LIST:
            ws.append(["", f"• {d_name}", "", "", "", "", "", "", "", "", ""])
            doc_row = ws.max_row
            excel_styler.apply_row_style(ws, doc_row, config.FONT_DATA, None, config.THIN_BORDER,
config.ALIGN_L)

            ws.cell(row=doc_row, column=3).number_format = '#,##0.00'
            ws.row_dimensions[doc_row].outline_level = 3

            mask = config.DOCUMENT_ROLES.get(d_name, {"СтрК": 1, "СД0": 1, "ГенДир": 1, "1 экз.
3.": 1, "1 экз. П": 1, "0пл.": 1})
            column_mapping = {"СтрК": 4, "СД0": 5, "ГенДир": 6, "1 экз. 3.": 7, "1 экз. П": 8,
"0пл.": 9}

            for col_name, col_idx in column_mapping.items():
                cell = ws.cell(row=doc_row, column=col_idx)
                if mask.get(col_name, 1) == 0:
                    cell.fill = config.FILL_BLOCKED
                    cell.value = ""
                else:
                    if col_name == "СтрК":
                        excel_styler.format_status_cell(cell, status_raw)

        # НОВЫЙ 5-Й УРОВЕНЬ: Если у документа в СтрК есть дата изменения, добавляем строку-
логгер
        if mask.get("СтрК", 1) == 1 and status_date:
            ws.append(["", f"    Дата изменения СтрК: {status_date}", "", "", "", "", "", "", "",
"", "", ""])

            log_row = ws.max_row
            # Применяем легкие границы и серый курсивный шрифт
            excel_styler.apply_row_style(ws, log_row, excel_styler.FONT_LOG_DATE, None,
config.THIN_BORDER, config.ALIGN_L)
            # Задаем уровень группировки 4 (в Excel это отобразится как 5-й уровень
вложенности)
            ws.row_dimensions[log_row].outline_level = 4

        for row in range(2, ws.max_row + 1):
            if ws.cell(row=row, column=2).value in config.MONTHS_LIST:
                ws.cell(row=row, column=10).value = f"=IF(C{row}>0, \"В работе\", \"\")"

        return ws.max_row

```

ФАЙЛ: excel_styler.py

```

import openpyxl
import config

# Настраиваем деликатный серый курсив для 5-го уровня истории дат
FONT_LOG_DATE = openpyxl.styles.Font(name="Calibri", size=9, italic=True, color="7A7A7A")

def apply_row_style(ws, row_idx, font, fill, border, alignment=None):
    """Применяет стили оформления к указанной строке таблицы."""
    for col in range(1, ws.max_column + 1):
        cell = ws.cell(row=row_idx, column=col)
        if font: cell.font = font
        if fill: cell.fill = fill
        if border: cell.border = border
        if alignment: cell.alignment = alignment

def format_status_cell(cell, status_raw):
    """Записывает в ячейку чистое число статуса для корректной работы условного цвета."""
    status_val = ""

    if isinstance(status_raw, dict):
        status_val = status_raw.get("value", "")

```

```

else:
    status_val = status_raw

if status_val == "" or status_val is None:
    return ""

try:
    # ВОЗВРАЩАЕМ ИСПРАВЛЕНИЕ: Преобразуем статус в число int, чтобы сработал цвет Excel
    cell.value = int(status_val)
except ValueError:
    cell.value = str(status_val)

cell.font = config.FONT_DATA
cell.alignment = config.ALIGN_C
return cell.value

```

ФАЙЛ: config.py

```

import os
from openpyxl.styles import Font, Alignment, PatternFill, Border, Side

DATA_ROWS = [
    ('01_АНАПА (Таманская)', '01_271_КН', 'Февраль', 450000, 3),
    ('01_АНАПА (Таманская)', '01_271_КН', 'Март', 0, None),
    ('01_АНАПА (Таманская)', '02_1505_КН.', 'Февраль', 450000, 3),
    ('01_АНАПА (Таманская)', '02_1505_КН.', 'Март', 0, None),
    ('01_АНАПА (Таманская)', '03_538_КН', 'Февраль', 0, None),
    ('01_АНАПА (Таманская)', '03_538_КН', 'Март', 0, None),
    ('01_АНАПА (Таманская)', '04_537_КН', 'Февраль', 0, None),
    ('01_АНАПА (Таманская)', '04_537_КН', 'Март', 0, None),
    ('02_ИНДУСТРИАЛЬНАЯ Краснодар', 'Основной участок', 'Февраль', 850000, 2),
    ('02_ИНДУСТРИАЛЬНАЯ Краснодар', 'Основной участок', 'Март', 0, None),
]

MONTHS_LIST = ['Февраль', 'Март', 'Июнь', 'Июль']
DOCUMENTS_LIST = ['Акт КС-2', 'Справка КС-3', 'Счет-фактура', 'Счет', 'Исполнительная документация']

FOLDER_NAME = ""
FILE_NAME = "Трекер_Акт_Выполнения.xlsx"
FULL_PATH = os.path.join(FOLDER_NAME, FILE_NAME)

FONT_HDR = Font(name="Calibri", size=11, bold=True, color="FFFFFF")
FONT_DIR = Font(name="Calibri", size=12, bold=True, color="FFFFFF")
FONT_OBJ = Font(name="Calibri", size=11, bold=True, color="000000")
FONT_MTH = Font(name="Calibri", size=11, bold=True, italic=True, color="000000")
FONT_DATA = Font(name="Calibri", size=11)

FILL_HDR = FILL_DIR = PatternFill(start_color="1F4E78", end_color="1F4E78", fill_type="solid")
FILL_OBJ = PatternFill(start_color="D9D9D9", end_color="D9D9D9", fill_type="solid")
FILL_MTH = PatternFill(start_color="EAEAEA", end_color="EAEAEA", fill_type="solid")

THIN_BORDER = Border(left=Side(style="thin", color="B0B0B0"), right=Side(style="thin", color="B0B0B0"),
top=Side(style="thin", color="B0B0B0"), bottom=Side(style="thin", color="B0B0B0"))

ALIGN_C = Alignment(horizontal="center", vertical="center", wrap_text=True)
ALIGN_L = Alignment(horizontal="left", vertical="center", wrap_text=True)
ALIGN_R = Alignment(horizontal="right", vertical="center")

HEADERS = ["№ п/п", "Наименование объекта / Месяц / Документ", "Сумма (руб.)", "СтрК", "СДО", "ГенДир",
"1 экз. 3.", "1 экз. П", "Опл.", "Текущий статус акта"]

# === МАСКИ ПОВЕДЕНИЯ (ТРАФАРЕТЫ ДЛЯ ДОКУМЕНТОВ) ===
# 1 – колонка активна (белая), 0 – заблокирована (серая заливка)
ROLE_KS2 = {"СтрК": 1, "СДО": 1, "ГенДир": 1, "1 экз. 3.": 1, "1 экз. П": 1, "Опл.": 0}
ROLE_KS3 = {"СтрК": 0, "СДО": 1, "ГенДир": 1, "1 экз. 3.": 1, "1 экз. П": 1, "Опл.": 0}
ROLE_INVOICE = {"СтрК": 0, "СДО": 1, "ГенДир": 0, "1 экз. 3.": 1, "1 экз. П": 1, "Опл.": 0}
ROLE_EXEC_DOC = {"СтрК": 1, "СДО": 0, "ГенДир": 0, "1 экз. 3.": 1, "1 экз. П": 1, "Опл.": 0}

DOCUMENT_ROLES = {
    "Акт КС-2": ROLE_KS2,
    "Справка КС-3": ROLE_KS3,
    "Счет-фактура": ROLE_INVOICE,
    "Счет": ROLE_INVOICE,
    "Исполнительная документация": ROLE_EXEC_DOC
}

# Пастельно-серый цвет для блокировки неиспользуемых ячеек
FILL_BLOCKED = PatternFill(start_color="EAEAEA", end_color="EAEAEA", fill_type="solid")

```



```
##### ФАЙЛ: git_deployer.py #####
```

```
import os
import json
import subprocess
import config
```

```
def get_current_git_version():
    try:
        res = subprocess.run(["git", "describe", "--tags", "--abbrev=0"], capture_output=True, text=True,
check=True)
        version = res.stdout.strip()
        if version.startswith('v'):
            parts = version[1:].split('.')
            parts[-1] = str(int(parts[-1]) + 1)
            return f"v{'.'.join(parts)}"
        return "v2.1"
    except Exception:
        return "v2.1"
```

```
def generate_auto_changelog(session_added_rows=None):
    if session_added_rows:
        directions = set([r[0] for r in session_added_rows])
        months = set([r[2] for r in session_added_rows])
        return f"Добавлены новые записи для: {'', '.join(directions)} ({', '.join(months)})"
```

```
json_path = "changes.json"
if os.path.exists(json_path) and os.path.getsize(json_path) > 2:
    try:
        with open(json_path, "r", encoding="utf-8") as f:
            changes = json.load(f)
        if changes:
            months_edited = []
            for k, v in changes.items():
                if "||" in k:
                    months_edited.append(k.split("||")[-1])
                elif isinstance(v, dict) and "label" in v:
                    months_edited.append(v["label"].strip())
            if months_edited:
                return f"Внесены ручные изменения сумм для месяцев: {'', '.join(set(months_edited))}"
    except Exception:
        pass
```

```
return "Плановое обновление структуры и формул трекера"
```

```
def clear_changes_json():
    json_path = "changes.json"
    if os.path.exists(json_path):
        try:
            with open(json_path, "w", encoding="utf-8") as f:
                json.dump({}, f)
            print(" [ОЧИСТКА] Буфер временных изменений changes.json успешно очищен.")
        except Exception as e:
            print(f" [ПРЕДУПРЕЖДЕНИЕ] Неудалось очистить changes.json: {e}")
```

```
def run_production_pipeline(session_added_rows=None):
    print("\n [ДВИЖОК] Шаг 1: Запуск локальной генерации таблицы...")
    result = subprocess.run(["python3", "main.py"])
    if result.returncode != 0:
        print(" [ОШИБКА] Сборка завершилась неудачно. Отправка в репозиторий заблокирована.")
        return False
```

```
print("\n [ДВИЖОК] Шаг 2: Запрос синхронизации с облаком")
push_choice = input("Выгрузить обновленный трекер в репозиторий GitHub? (да/нет): ").strip().lower()
```

```
if push_choice in ("да", "y", "yes"):
    default_ver = get_current_git_version()
    default_logs = generate_auto_changelog(session_added_rows)

    print("\n Настройка метаданных коммита:")
    user_ver = input(f"Номер версии [По умолчанию: {default_ver}]: ").strip()
    final_ver = user_ver if user_ver else default_ver

    user_logs = input(f"Описание изменений [По умолчанию: {default_logs}]: ").strip()
    final_logs = user_logs if user_logs else default_logs

    commit_message = f"{final_ver} - {final_logs}"
```

```

print(f"\n [GIT] Будет отправлено с описанием: \"{commit_message}\"")

subprocess.run(["git", "add", "."])
subprocess.run(["git", "commit", "-m", commit_message])
subprocess.run(["git", "tag", "-a", final_ver, "-m", final_logs])
push_res = subprocess.run(["git", "push", "origin", "main", "--tags"])

if push_res.returncode == 0:
    print(" [УСПЕХ] Все файлы и тег версии успешно отправлены на GitHub!")
    clear_changes_json()
    return True
else:
    print(" [ОШИБКА GIT] Не удалось отправить файлы в облако. Проверьте сеть или доступы.")
    return False
else:
    print(" [ИНФО] Изменения сохранены только локально на Mac. Выгрузка отменена.")
    clear_changes_json()
    return True

```

ФАЙЛ: main.py

```

import os
import openpyxl
import config
import storage
import builder
import formatter

def run():
    print("🌀 [Автоматический запуск] Сборка смарт-трекера...")
    file_path = config.FULL_PATH if config.FULL_PATH else "Трекер_Акт_Выполнения.xlsx"

    # 1. Загрузка истории
    saved_statuses, saved_sums = storage.load_historical_data(file_path)
    mock_data = storage.group_linear_data(config.DATA_ROWS)

    # 2. Инициализация книги
    wb = openpyxl.Workbook()
    ws = wb.active
    ws.title = "Реестр Акт"
    ws.sheet_view.showGridLines = True
    ws.sheet_view.showOutlineSymbols = True
    ws.sheet_properties.outlinePr.summaryBelow = False

    # 3. Шапка
    ws.append(config.HEADERS)
    ws.row_dimensions.height = 28
    builder.apply_row_style(ws, 1, config.FONT_HDR, config.FILL_HDR, config.THIN_BORDER, config.ALIGN_C)

    # 4. Построение и стилизация
    last_row = builder.build_structure(ws, mock_data, saved_statuses, saved_sums)
    formatter.append_specifications(ws, last_row)
    formatter.apply_conditional_formatting(ws)
    formatter.set_column_widths(ws)

    # 5. Безопасное сохранение
    try:
        wb.save(file_path)

        # ФИЗИЧЕСКАЯ ПРОВЕРКА НАЛИЧИЯ ФАЙЛА НА ДИСКЕ
        if os.path.exists(file_path) and os.path.getsize(file_path) > 0:
            file_size_kb = round(os.path.getsize(file_path) / 1024, 1)
            print(f"📡 [ДАТЧИК КОНТРОЛЯ] Файл успешно обнаружен в системе!")
            print(f"📍 Путь: {os.path.abspath(file_path)}")
            print(f"⚖️ Вес документа: {file_size_kb} КБ")
            print(f"🚀 [УСПЕХ] Таблица полностью обновлена автоматически!")
        else:
            print("🚫 [ВНИМАНИЕ] Файл записан, но датчик системы указывает на пустой или отсутствующий документ!")

    except PermissionError:
        print("❌ [ОШИБКА ДОСТУПА] Файл Excel/LibreOffice открыт! Закройте его для генерации.")
    except Exception as e:
        print(f"❌ [КРИТИЧЕСКИЙ СБОЙ] Запись сорвалась: {e}")

```



```
if __name__ == "__main__":
    run()
```

```
### ФАЙЛ: database.json ###
```

```
{
    "01_АНАПА (Таманская)": {
        "01_271_КН": {
            "Июнь": {
                "sum": 17730436.92,
                "status": {
                    "value": 2,
                    "date": "01.07"
                }
            },
            "Июль": {
                "sum": 0.0,
                "status": ""
            }
        },
        "02_1505_КН.": {
            "Февраль": {
                "sum": 450000,
                "status": 3
            },
            "Март": {
                "sum": 0,
                "status": ""
            }
        },
        "03_538_КН": {
            "Февраль": {
                "sum": 0,
                "status": ""
            },
            "Март": {
                "sum": 0,
                "status": ""
            }
        },
        "04_537_КН": {
            "Февраль": {
                "sum": 0,
                "status": ""
            },
            "Март": {
                "sum": 0,
                "status": ""
            }
        }
    },
    "02_ИНДУСТРИАЛЬНАЯ Краснодар": {
        "Основной участок": {
            "Февраль": {
                "sum": 850000,
                "status": 2
            },
            "Март": {
                "sum": 0,
                "status": ""
            }
        }
    }
}
```
